SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

Mode: In Class

Total Marks:50

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

88

Code of Conduct:

(192472250).

I S Deva Dhasshini *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

S Deva...

Signature of the student:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ical Analysis:

1924178550.
S. Deva Dhashini

Lexical Analysis is the first phase of the compiler. it reads the input characters by characters and converts it into tokens.

a) Tokens generated:.

Lexeme	Token
p	Identifier
=	Assignment operator
(	left parenthesis
a	Identifier
*	multiplication operator
b	Identifier
)	Right parenthesis
+	Addition operator
C	left parenthesis
b	Identifier
*	multiplication operator
c	Identifier
)	Right parenthesis
*	multiplication operator
2	Numeric constant

1 b) Syntax Analysis/ Parsing

The parser checks whether the token sequence follows the grammar of the languge and constructs a syntax tree.

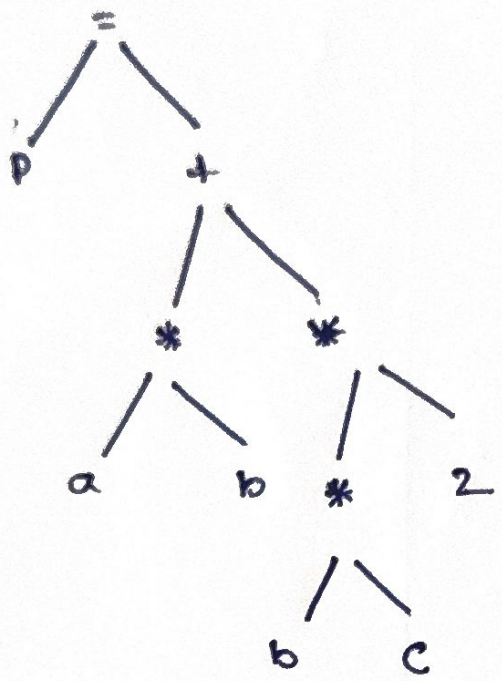
The Expression is interpreted according to operator precedence.

- Parantheses have highest priority.
- multiplication * has higher priority than addition +.

Therefore:

P = (a*b) + (b*c)*2).

Representation of Syntax tree:



main Expression is

(a*b) + ((b*c) *2)

1. c) Intermediate code generation.

The compiler converts the syntax tree into an intermediate representation (IR). A common form is Three Address code.

Intermediate code

For:

P = (a*b) + (b*c)*2)

We generate:

t1 = a*b

t2 = b*c

t3 = t2*2

t4 = t1+t3

P = t4

Here t1, t2, t3 and t4 are temporary variable.

...te optimaization.

ptimaization tries to improve the intermediate code without changing its meaning.

original:

```
t1 = a * b
t2 = b * c
t3 = t2 * 2
t4 = t1 + t3
P = t4.
```

optimaized:

```
t1 = a * b
t2 = b * c
t2 = t2 * 2
P = t1 + t2.
```

The unnecessary temporary variables t3 and t4 are eliminated.

e) Target code generation:-

The final phase converts the optimal intermediate code into target/machine code suitable for the target processor.

Target code:

```
load  R1, a
MUL   R1, b
load  R2, b
MUL   R2, c
MUL   R2, 2
Add   R1, R2
STORE P, R1
```

8. Regular Expressions.

For Each regular expression, construct the language and write valid

a) $(\varepsilon + aa^*)^*$

Here

aa^* means one (or) more a is

so the expression generates any number of a's, including the empty string.

language :

$$L = \{a^n \mid n \geq 0\}$$

valid strings (few):

1. ε
2. a
3. aa
4. aaa
5. aaaa

b) $(a^* + b^*)^* abb$

$(a^* + b^*)^*$ can generate any combination of a and b.

Then abb must occur at the end.

So the language consists of all strings over {a, b} ending with abb.

few valid strings:

1. abb
2. aabb
3. babb
4. ababb
5. bbabb

c) $(a^* b)^* aba (a^* + b^*)^*$

The expression contains aba and the parts before and after can generate strings of a and b.

∴ The language contains string having aba as a substring.

1. aba
2. aaba

aa

baba

5. ababa

d. $(a+ab)^*$

Each block can be either

- a
- ab

∴ The language is formed by concatenating any number of a (or) ab blocks.

Five valid strings.

1. ε
2. a
3. aa
4. ab
5. aab

aab = a+ab.

e) $(aba)^* + (a^*b^*)^*$

The second part $(a^*b^*)^*$ can generate any string containing a and b.

∴ Language

$$L = \{a,b\}^*$$

Five valid strings

1. ε
2. a
3. b
4. ab
5. ba.

③ Lexical Analysis.

Given

to find the no. of tokens in the following C statement.

```
main ()
{
int a=10;
char b='abc';
int c=30;
```

```
char d = "xyz";
int m = 10;
}
```

Token by token counting:

S.no	token
1	main
2	(
3	)
4	{
5	int
6	a
7	=
8	10
9	;
10	char
11	b
12	"abc"
13	;
14	int
15	c
16	=
17	30
18	;
19	char
20	d
21	=
22	"xyz"
23	;
24	int
25	m
26	=

7 to 5

28 ;

29 }

Answer.

29 tokens.

(ii) (iii)

```
void main(){
  int x=10, y=20;
   char *a;
  *a= &x;
   x = 1xab;
}
```

Error identification

X = 1Xab;

1Xab is an invalid lexical token because an identifier cannot begin with digit.

It starts with 1 and is followed by letters (Xab), so it is not a valid integer constant (or) a valid identifier.

correct form:-

X = Xab

Provided Xab has been declared.

Answer:

lexical error: 1Xab; An identifier cannot start with a digit.

(4)

(i) Strings having an even number of 0 s

We can have any number of 1 s between pairs of 0s

Regular Expression

$1^*(01^*01^*)^*$

Examples : ε, 1, 00, 11, 0101, 0011

(iii) Strings containing atleast one 1.

There can be anything before and after atleast one 1

Regular expression

$$(0+1)^* 1 (0+1)^*$$

Examples: 1, 01, 10, 101, 00101 00

(iii) Strings in which both the numbers of 0s and the number of 1's are odd

A way to express is

$$0(00)^* 1 (11)^* (01(00)^* 1 (11)^* 01(00)^* 1(11)^*)^*$$

valid strings: 01, 10, 0001, 0111, 10600

(vi) string that start with 1 and end with 0

The middle can contain any combination of 0 and 1.

$$1(0+1)^* 0$$

Ex: 10, 100, 110, 1010, 1110.

(v) strings that do not contain consecutive 1s.

A 1 must either be followed by 0 (or) occur at the end.

$$(0+10)^* (\varepsilon+1).$$

Ex:

ε, 0, 1, 10, 101, 010, 1010.

5

(a) Number of tokens.

int

val 123

=

12345

;

float

val 456

=

val

val456

=

123.45

;

if

(

val123

==

val456

)

{

return

;

}

Therefore

20 Tokens

b) keywords.

int, float, if, return.

operators.

=, ==, =

separators

;, ,(,), {, }, ;

Identifiers

val123, val456, val123, val456.

Integer literals

12345

Floating-point literal

123.45.

∴ 20 tokens.
